

PRINCE TOGGLE

REALISATION

MISE EN PLACE D'UN SIEM (SÉCURITÉ & DÉTECTION DES INCIDENTS)

WAZUH



SOMMAIRE

1. Présentation et architecture
2. Prérequis et configuration réseau
3. Installation de Wazuh (all-in-one)
4. Configuration de base
5. Intégration avec Active Directory (LDAP/LDAPS)
6. Installation des agents Wazuh
7. Configuration des règles personnalisées
8. Tests et validation
9. Livrables pour la Mission 10
10. Dépannage

1. PRÉSENTATION ET ARCHITECTURE

1.1 Qu'est-ce que Wazuh ?

Wazuh est une plateforme de sécurité open source qui combine EDR/HIDS et SIEM. Elle permet de :

- Collecter et analyser les logs (sécurité, système, applications)
- Détecter les intrusions et comportements anormaux
- Surveiller l'intégrité des fichiers
- Évaluer les vulnérabilités
- Répondre automatiquement aux incidents

1.2 Architecture cible

Architecture Wazuh  		
Composant	Rôle	IP / Emplacement
Wazuh Manager	Serveur central (analyse + gestion)	172.20.1.30 (VLAN 10)
Wazuh Indexer	Stockage et indexation des alertes	Même VM (all-in-one)
Wazuh Dashboard	Interface web	Même VM (all-in-one)
Active Directory	Authentification centralisée	Hermès (172.20.1.2)
Agents Wazuh	Collecte sur les machines à superviser	Tous les serveurs

2. PRÉREQUIS ET CONFIGURATION RÉSEAU

2.1 Fiche technique

Fiche technique  	
Élément	Valeur recommandée
Système d'exploitation	Ubuntu 22.04 LTS ou Debian 12
Hostname	wazuh
FQDN	wazuh.stadiumcompany.local
Adresse IP	172.20.1.30/24 (fixe)
Passerelle	172.20.1.254 (pfSense)
DNS	172.20.1.2 (Hermès), 172.20.1.14 (Ares)
RAM	8 Go minimum (16 Go recommandé)
CPU	4 cœurs
Disque	100 Go minimum (indexation)

2.2 Création de la VM

- Crée une VM avec Ubuntu 22.04 LTS.
- Configure 2 cartes réseau :
 - ens33 : VLAN 10 (administration) → IP fixe 172.20.1.30
 - ens34 : Accès Internet (NAT) pour les mises à jour
- Monte l'ISO Ubuntu 22.04 et installe le système.

2.3 Configuration réseau

```
# Passer en root
sudo su
```

```
# Configuration de l'interface réseau
nano /etc/netplan/00-installer-config.yaml
```

```
user@wazuh:~$ sudo su
[sudo] password for user:
root@wazuh:/home/user# nano /etc/netplan/50-cloud-init.yaml
```

Contenu :

```
network:
  version: 2
  ethernet:
    ens33:
      addresses:
        - 172.20.1.30/24
      routes:
        - to: default
          via: 172.20.1.254
      nameservers:
        addresses:
          - 172.20.1.2
          - 172.20.1.14
        search:
          - stadiumcompany.local
    ens34:
      dhcp4: true
```

```
network:
  ethernet:
    ens34:
      dhcp4: true
    ens33:
      addresses:
        - 172.20.1.30/24
      routes:
        - to: default
          via: 172.20.1.254
      nameservers:
        addresses:
          - 172.20.1.2
          - 172.20.1.14
        search:
          - stadiumcompany.local
  version: 2
```

Application :

```
netplan apply
```

2.4 Configuration DNS

```
# Vérification
ping hermes.stadiumcompany.local
nslookup hermes.stadiumcompany.local
```

```

root@wazuh:/home/user# ping hermes.stadiumcompany.local
nslookup hermes.stadiumcompany.local
PING hermes.stadiumcompany.local (172.20.1.2) 56(84) bytes of data.
64 bytes from hermes.stadiumcompany.local (172.20.1.2): icmp_seq=1 ttl=128 time=0.533 ms
64 bytes from hermes.stadiumcompany.local (172.20.1.2): icmp_seq=2 ttl=128 time=1.04 ms
64 bytes from hermes.stadiumcompany.local (172.20.1.2): icmp_seq=3 ttl=128 time=0.619 ms
64 bytes from hermes.stadiumcompany.local (172.20.1.2): icmp_seq=4 ttl=128 time=0.959 ms
64 bytes from hermes.stadiumcompany.local (172.20.1.2): icmp_seq=5 ttl=128 time=1.21 ms
64 bytes from hermes.stadiumcompany.local (172.20.1.2): icmp_seq=6 ttl=128 time=1.71 ms
^C
--- hermes.stadiumcompany.local ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 0.533/1.011/1.708/0.389 ms
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   hermes.stadiumcompany.local
Address: 172.20.1.2

root@wazuh:/home/user#

```

2.5 Mise à jour du système

```
apt update && apt upgrade -y
```

3. INSTALLATION DE WAZUH (ALL-IN-ONE)

3.1 Installation des prérequis

```
apt install -y curl wget gnupg apt-transport-https software-properties-common
```

```

root@wazuh:/home/user# apt install -y curl wget gnupg apt-transport-https software-properties-common
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
curl est déjà la version la plus récente (7.81.0-1ubuntu1.22).
curl passé en « installé manuellement ».
gnupg est déjà la version la plus récente (2.2.27-3ubuntu2.5).
gnupg passé en « installé manuellement ».
software-properties-common est déjà la version la plus récente (0.99.22.9).
software-properties-common passé en « installé manuellement ».
wget est déjà la version la plus récente (1.21.2-2ubuntu1.1).
wget passé en « installé manuellement ».

```

3.2 Téléchargement et exécution de l'assistant

```

# Télécharger le script d'installation
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh

```

```

# Rendre le script exécutable
chmod +x wazuh-install.sh

```

```

# Lancer l'installation (option -a pour all-in-one)
bash wazuh-install.sh -a

```

```
Yr*b7XgFeaMTUsw+2s5fs.qPniSh9d*P
```

```
08/03/2026 20:52:57 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/0
08/03/2026 20:53:09 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore usern
08/03/2026 20:53:37 INFO: Initializing Wazuh dashboard web application.
08/03/2026 20:53:38 INFO: Wazuh dashboard web application initialized.
08/03/2026 20:53:38 INFO: --- Summary ---
08/03/2026 20:53:38 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
    User: admin
    Password: Yr*b7XgFeaMTUsw+2s5fs.qPniSh9d*P
08/03/2026 20:53:38 INFO: Installation finished.
root@wazuh:/home/user#
```

L'installation prend environ 5-10 minutes.

3.3 Récupération des identifiants

À la fin de l'installation, l'assistant affiche :

```
INFO: --- Summary ---
INFO: You can access the web interface https://<wazuh-dashboard-ip>
User: admin
Password: <ADMIN_PASSWORD>
INFO: Installation finished.
```

⚠ **Note ces informations !** Le mot de passe est généré aléatoirement.

Si tu perds le mot de passe :

```
bash wazuh-install.sh -p
```

3.4 Vérification des services

```
systemctl status wazuh-manager
systemctl status wazuh-indexer
systemctl status wazuh-dashboard
```

```
root@wazuh:/home/user# systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-03-08 20:53:25 UTC; 3min 10s ago
     Tasks: 173 (limit: 9344)
    Memory: 2.5G
       CPU: 36.781s
    CGroup: /system.slice/wazuh-manager.service
```

```
root@wazuh:/home/user# systemctl status wazuh-indexer
● wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/lib/systemd/system/wazuh-indexer.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-03-08 20:50:06 UTC; 6min ago
     Docs: https://documentation.wazuh.com
    Main PID: 26787 (java)
     Tasks: 84 (limit: 9344)
    Memory: 1.3G
       CPU: 1min 3.615s
```

```
root@wazuh:/home/user# systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-03-08 20:53:26 UTC; 3min 43s ago
    Main PID: 81616 (node)
     Tasks: 11 (limit: 9344)
    Memory: 177.5M
       CPU: 7.895s
```

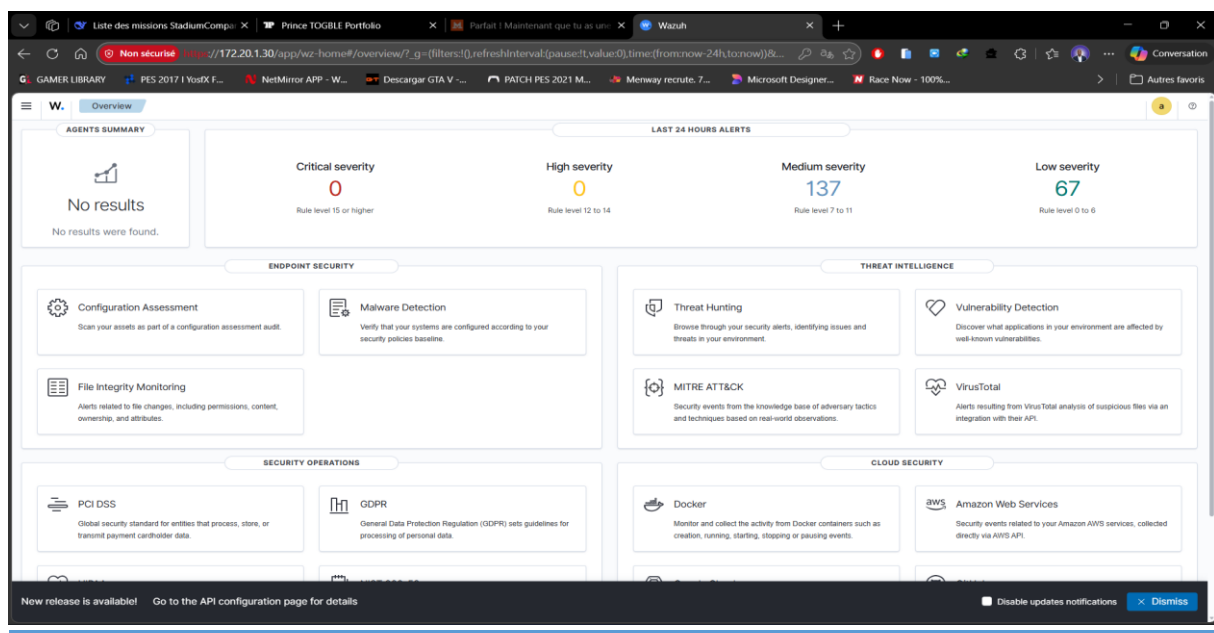
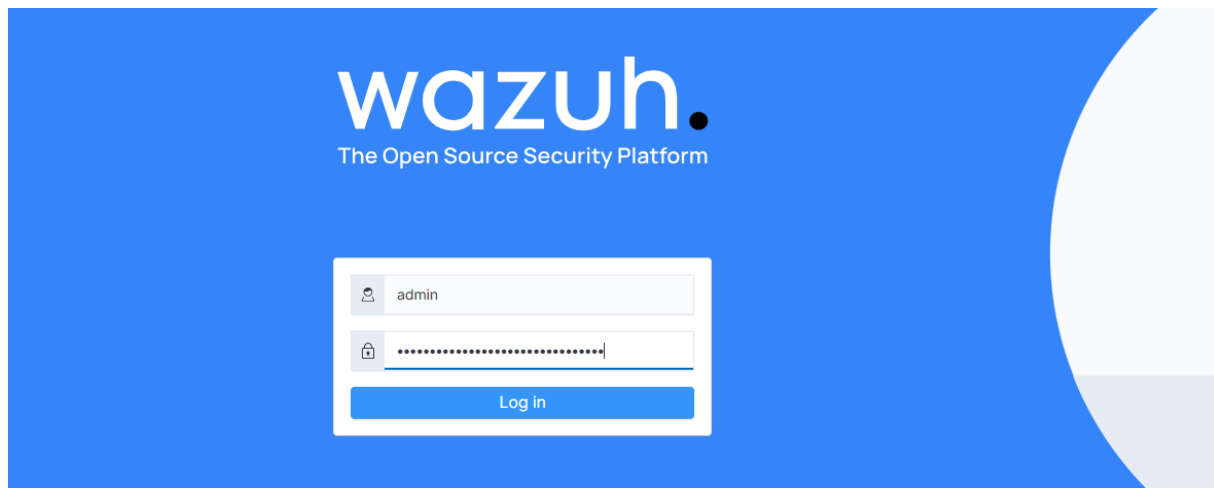
3.5 Configuration du pare-feu

```
ufw allow 22/tcp           # SSH
ufw allow 443/tcp          # Dashboard HTTPS
ufw allow 1514/tcp         # Agent TCP
ufw allow 1514/udp         # Agent UDP
ufw allow 1515/tcp         # Agent enrollment
ufw allow 55000/tcp        # Wazuh API
ufw enable
```

4. CONFIGURATION DE BASE

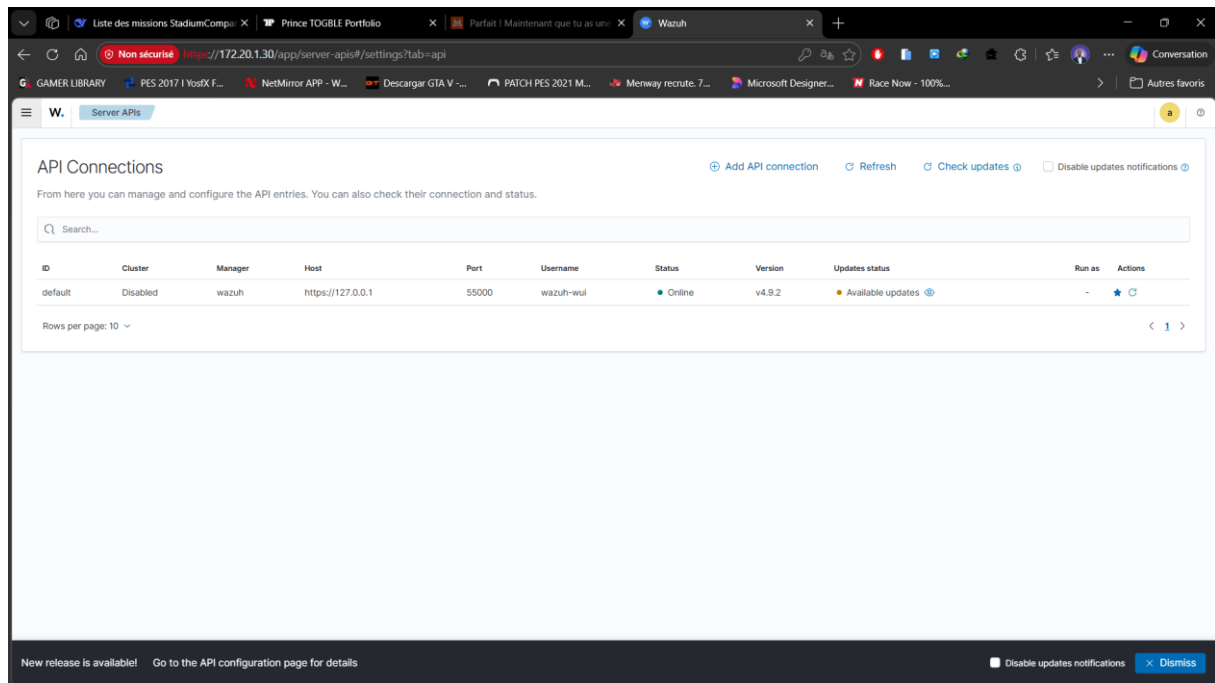
4.1 Premier accès au Dashboard

- Depuis un navigateur (VLAN 10) : <https://172.20.1.30>
- Accepte le certificat auto-signé (temporaire)
- Connecte-toi avec les identifiants **admin**



4.2 Configuration de l'API

- Wazuh Dashboard → **Settings** → **API configuration** → Vérifie que l'API est accessible.



4.3 Vérification de l'état

- Va dans **Modules** → **Status** pour vérifier que tous les composants sont verts.

5. INTÉGRATION AVEC ACTIVE DIRECTORY (LDAP/LDAPS)

5.1 Préparation côté Active Directory (Hermès)

- Créer le compte de service wazuhad :**
 - Utilisateurs et ordinateurs Active Directory → **Users** → **Nouveau** → **Utilisateur**
 - Nom : **wazuh**
 - Nom d'utilisateur : **wazuhad**
 - Mot de passe : **Bts2024@**

Nouvel objet - Utilisateur

Créer dans : stadiumcompany.local/Users

Prénom : wazuh Initiales :

Nom :

Nom complet : wazuh

Nom d'ouverture de session de l'utilisateur : wazuhad @stadiumcompany.local

Nom d'ouverture de session de l'utilisateur (antérieur à Windows 2000) : STADIUMCOMPANY\ wazuhad

< Précédent Suivant > Annuler

- Cocher : "Le mot de passe n'expire jamais"
- **Créer le groupe G_WAZUH_ADMIN :**
 - Nouveau → **Groupe**
 - Nom : **G_WAZUH_ADMIN**
 - Portée : **Globale**
 - Type : **Sécurité**
 - Ajouter les utilisateurs concernés (ex: Administrateur) dans ce groupe

Nouvel objet - Groupe

Créer dans : stadiumcompany.local/Users

Nom du groupe : G_WAZUH_ADMIN

Nom de groupe (antérieur à Windows 2000) : G_WAZUH_ADMIN

Étendue du groupe

☐ Domaine local

☒ Globale

☐ Universelle

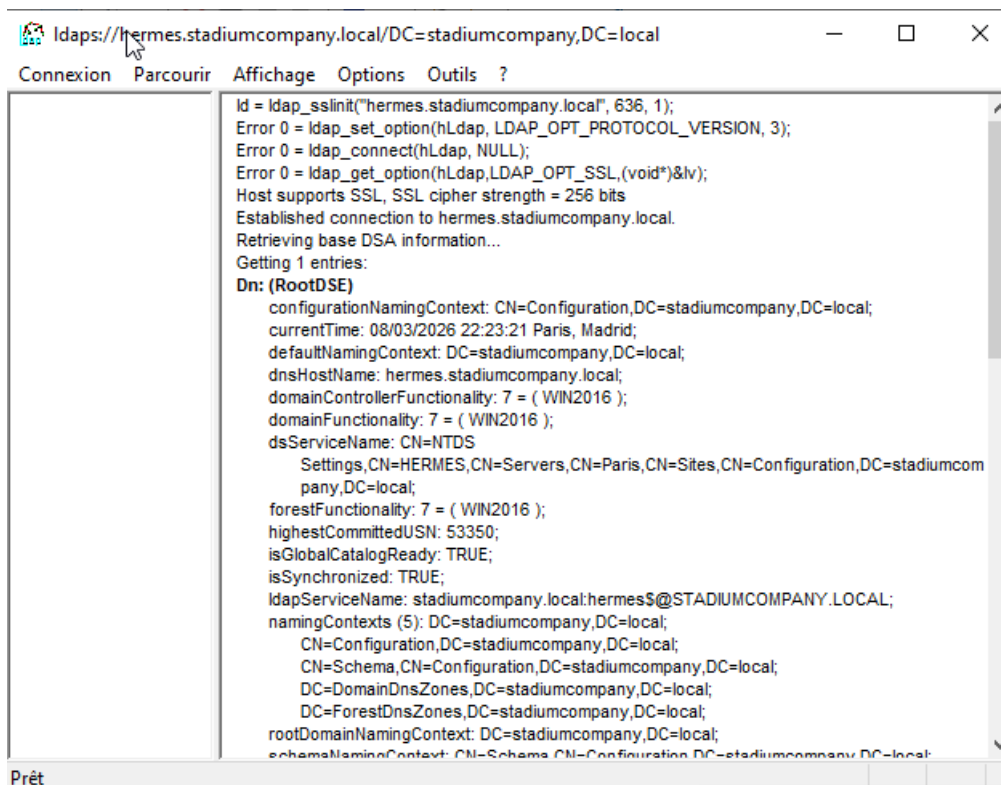
Type de groupe

☒ Sécurité

☐ Distribution

OK Annuler

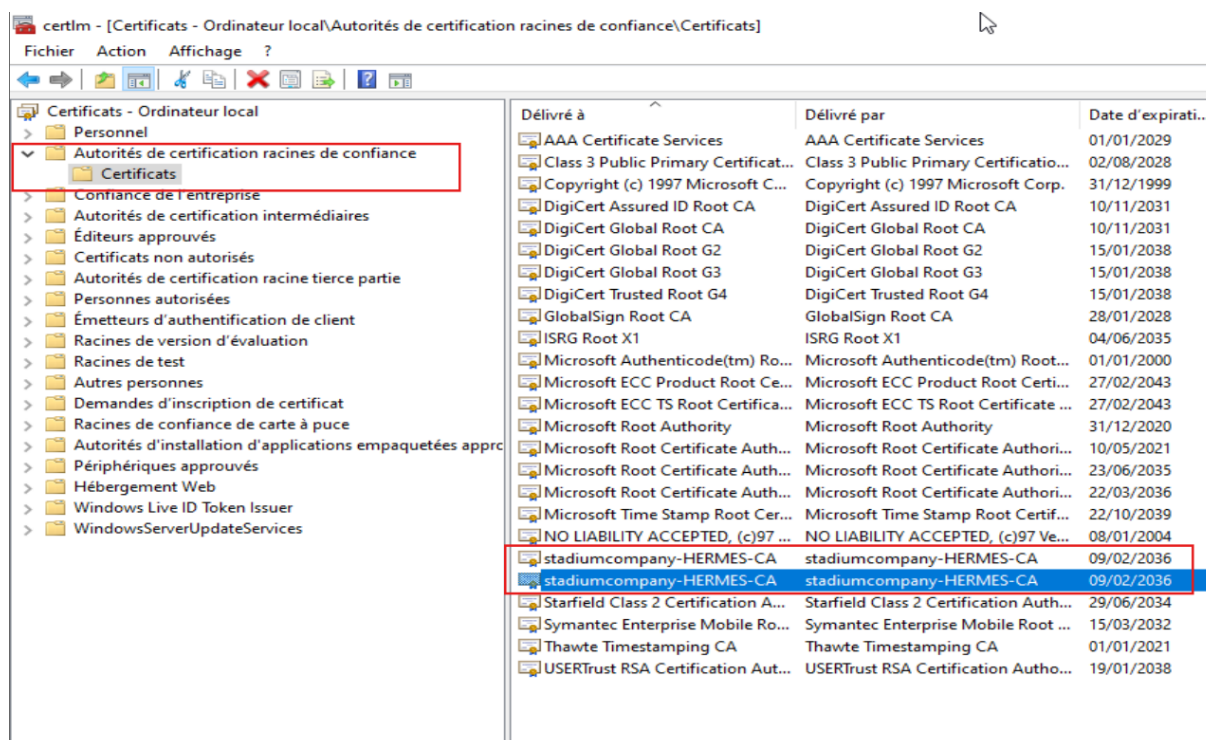
- **Vérifier LDAPS :**
- # Depuis Hermès, vérifier que LDAPS (636) fonctionne
ldp.exe → Connexion → Connect → Serveur :
hermes.stadiumcompany.local, Port : 636, SSL coché



5.2 Récupérer le certificat racine AD

Sur Hermès

certlm.msc → Autorités de certification racines de confiance → Certificats → Trouver le certificat "Hermes-AD-CA" (ou stadiumcompany-HERMES-CA) → Clic droit → Exporter → Format : Base-64 encoded X.509 (.cer) → Enregistrer (ex: AD-RootCA.cer)



Transfère ce fichier vers le serveur Wazuh (via SCP, clé USB, etc.).

5.3 Sur le serveur Wazuh : configuration LDAP

Étape 1 : Placer le certificat

```
# Créer le dossier de destination
mkdir -p /etc/wazuh-indexer/certs/ldap

# Copier le certificat
cp /mnt/hgfs/partage/AD-RootCA.cer /etc/wazuh-indexer/certs/ldap/ldapcacert.pem

# Définir les permissions
chown wazuh-indexer:wazuh-indexer /etc/wazuh-indexer/certs/ldap/ldapcacert.pem
chmod 400 /etc/wazuh-indexer/certs/ldap/ldapcacert.pem
```

```
root@wazuh:/home/user# ls /mnt/hgfs/
partage
root@wazuh:/home/user# ls -l /mnt/hgfs/partage/*.cer
-rwxrwxrwx 1 root root 1316 mars  8 21:31 /mnt/hgfs/partage/AD-RootCA.cer
root@wazuh:/home/user# cp /mnt/hgfs/partage/AD-RootCA.cer /etc/wazuh-indexer/certs/ldap/ldapcacert.pem
root@wazuh:/home/user# chown wazuh-indexer:wazuh-indexer /etc/wazuh-indexer/certs/ldap/ldapcacert.pem
root@wazuh:/home/user# chmod 400 /etc/wazuh-indexer/certs/ldap/ldapcacert.pem
root@wazuh:/home/user#
```

Étape 2 : Sauvegarder la configuration actuelle

```
cp /etc/wazuh-indexer/opensearch-security/config.yml /etc/wazuh-indexer/opensearch-security/config.yml.bak
```

```
root@wazuh:/home/user# cp /etc/wazuh-indexer/opensearch-security/config.yml /etc/wazuh-indexer/opensearch-security/config.yml.bak
root@wazuh:/home/user#
```

Étape 3 : Modifier la configuration LDAP

```
nano /etc/wazuh-indexer/opensearch-security/config.yml
```

Ajoute/modifie les sections **authc** et **authz** :

```
config:
  dynamic:
    authc:
      ldap:
        description: "Authentification via Active Directory StadiumCompany"
        http_enabled: true
        transport_enabled: false
        order: 5
        http_authenticator:
          type: basic
          challenge: false
        authentication_backend:
          type: ldap
          config:
            enable_ssl: true
```

```

    pemtrustedcas_filepath: /etc/wazuh-
indexer/certs/ldap/ldapcacert.pem
    enable_start_tls: false
    enable_ssl_client_auth: false
    verify_hostnames: true
    hosts:
      - "hermes.stadiumcompany.local:636"
    bind_dn: "cn=wazuhad,cn=Users,dc=stadiumcompany,dc=local"
    password: "@azerty1234"
    userbase: "cn=Users,dc=stadiumcompany,dc=local"
    usersearch: "(sAMAccountName={0})"
    username_attribute: sAMAccountName
  authz:
    roles_from_ad:
      description: "Authorisation via Active Directory"
      http_enabled: true
      transport_enabled: true
      authorization_backend:
        type: ldap
        config:
          enable_ssl: true
          pemtrustedcas_filepath: /etc/wazuh-
indexer/certs/ldap/ldapcacert.pem
          enable_start_tls: false
          enable_ssl_client_auth: false
          verify_hostnames: true
          hosts:
            - "hermes.stadiumcompany.local:636"
          bind_dn: "cn=wazuhad,cn=Users,dc=stadiumcompany,dc=local"
          password: "Bts2024@"
          userbase: "cn=Users,dc=stadiumcompany,dc=local"
          usersearch: "(sAMAccountName={0})"
          username_attribute: sAMAccountName
          rolebase: "cn=Users,dc=stadiumcompany,dc=local"
          rolesearch: "(member={0})"
          userrolename: memberOf
          rolename: cn
          skip_users:
            - admin
            - kibanaserver

```

```

ldap:
  description: "Authenticate via Active Directory Stadiumcompany"
  http_enabled: false
  transport_enabled: false
  order: 5
  http_authenticator:
    type: ldap
    challenge: false
  authentication_backend:
    # LDAP authentication backend (authenticate users against a LDAP or Active Directory)
    type: ldap
    config:

```

```
# Enable ldap
enable_ssl: true
pemtrustedcas_filepath: /etc/wazuh-indexer/certs/ldap/ldapcert.pem
# enable start tls, enable_ssl should be false
enable_start_tls: false
# send client certificate
enable_ssl_client_auth: false
# verify ldap hostname
verify_hostnames: true
hosts:
  - "hermes.stadiumcompany.local:636"
bind_dn: "cn=wazuhad,cn=Users,dc=stadiumcompany,dc=local"
password: "@azerty1234"
userbase: "cn=Users,dc=stadiumcompany,dc=local"
# Filter to search for users (currently in the whole subtree beneath userbase)
# {0} is substituted with the username
usersearch: '(sAMAccountName={0})'
# Use this attribute from the user as username (if not set then DN is used)
username_attribute: sAMAccountName
```

```
roles_from_ad:
  description: "Authorisation via Active Directory"
  http_enabled: true
  transport_enabled: true
  authorization_backend:
    type: ldap
    config:
      enable_ssl: true
      pemtrustedcas_filepath: /etc/wazuh-indexer/certs/ldap/ldapcert.pem
      enable_start_tls: false
      enable_ssl_client_auth: false
      verify_hostnames: true
      hosts:
        - "hermes.stadiumcompany.local:636"
      bind_dn: "cn=wazuhad,cn=Users,dc=stadiumcompany,dc=local"
      password: "Bts2024@"
      userbase: "cn=Users,dc=stadiumcompany,dc=local"
      usersearch: "(sAMAccountName={0})"
      username_attribute: sAMAccountName
      rolebase: "cn=Users,dc=stadiumcompany,dc=local"
      rolesearch: "(member={0})"
      userrolename: memberOf
      rolename: cn
      skip_users:
        - admin
        - kibanaserver
```

Étape 4 : Appliquer la configuration

```
export JAVA_HOME=/usr/share/wazuh-indexer/jdk/
```

```
bash /usr/share/wazuh-indexer/plugins/opensearch-
security/tools/securityadmin.sh \
```

```
-f /etc/wazuh-indexer/opensearch-security/config.yml \
```

```
-icl -key /etc/wazuh-indexer/certs/admin-key.pem \
```

```
-cert /etc/wazuh-indexer/certs/admin.pem \
```

```
-cacert /etc/wazuh-indexer/certs/root-ca.pem \
```

```
-h localhost -nhnv
```

```

root@wazuh:/home/user# export JAVA_HOME=/usr/share/wazuh-indexer/jdk/
bash /usr/share/wazuh-indexer/plugins/opensearch-security/tools/securityadmin.sh \
-f /etc/wazuh-indexer/opensearch-security/config.yml \
-icl -key /etc/wazuh-indexer/certs/admin-key.pem \
-cert /etc/wazuh-indexer/certs/admin.pem \
-cacert /etc/wazuh-indexer/certs/root-ca.pem \
-h localhost -nhnv
*****
** This tool will be deprecated in the next major release of OpenSearch **
** https://github.com/opensearch-project/security/issues/1755 **
*****
Security Admin v7
Will connect to localhost:9200 ... done
Connected as "CN=admin,OU=Wazuh,O=Wazuh,L=California,C=US"
OpenSearch Version: 2.13.0
Contacting opensearch cluster 'opensearch' and wait for YELLOW clusterstate ...
Clustername: wazuh-cluster
Clusterstate: GREEN
Number of nodes: 1
Number of data nodes: 1
.opendistro_security index already exists, so we do not need to create one.
Populate config from /home/user
Will update '/config' with /etc/wazuh-indexer/opensearch-security/config.yml
SUCC: Configuration for 'config' created or updated
SUCC: Expected 1 config types for node {"updated_config_types":["config"],"updated_config_size":1,"message":null} is 1 (["confi
Done with success
root@wazuh:/home/user# ^C
root@wazuh:/home/user#

```

Résultat attendu : SUCC: Configuration for 'config' created or updated.

5.4 Mapper les rôles LDAP aux rôles Wazuh

Étape 1 : Modifier le mapping des rôles

```
nano /etc/wazuh-indexer/opensearch-security/roles_mapping.yml
```

Ajoute :

```

all_access:
  reserved: false
  hidden: false
  backend_roles:
    - "admin"
    - "G_WAZUH_ADMIN"
  description: "Maps AD admin group to all_access"

```

```

all_access:
  reserved: false
  hidden: false
  backend_roles:
    - "admin"
    - "G_WAZUH_ADMIN"
  description: "Maps AD admin group to all_access"
  hosts: []
  users: []
  and_backend_roles: []
  description: "Maps admin to all_access"

```

Étape 2 : Appliquer le mapping

```

bash /usr/share/wazuh-indexer/plugins/opensearch-
security/tools/securityadmin.sh \
-f /etc/wazuh-indexer/opensearch-security/roles_mapping.yml \
-icl -key /etc/wazuh-indexer/certs/admin-key.pem \
-cert /etc/wazuh-indexer/certs/admin.pem \
-cacert /etc/wazuh-indexer/certs/root-ca.pem \
-h localhost -nhnv

```

```

root@wazuh:/home/user# nano /etc/wazuh-indexer/opensearch-security/roles_mapping.yml
root@wazuh:/home/user# bash /usr/share/wazuh-indexer/plugins/opensearch-security/tools/securityadmin.sh \
-f /etc/wazuh-indexer/opensearch-security/roles_mapping.yml \
-icl -key /etc/wazuh-indexer/certs/admin-key.pem \
-cert /etc/wazuh-indexer/certs/admin.pem \
-cacert /etc/wazuh-indexer/certs/root-ca.pem \
-h localhost -nhnv
*****
** This tool will be deprecated in the next major release of OpenSearch **
** https://github.com/opensearch-project/security/issues/1755 **
*****
Security Admin v7
Will connect to localhost:9200 ... done
Connected as "CN=admin,OU=Wazuh,O=Wazuh,L=California,C=US"
OpenSearch Version: 2.13.0
Contacting opensearch cluster 'opensearch' and wait for YELLOW clusterstate ...
Clustername: wazuh-cluster
Clusterstate: GREEN
Number of nodes: 1
Number of data nodes: 1
.opendistro_security index already exists, so we do not need to create one.
Populate config from /home/user
Will update '/rolesmapping' with /etc/wazuh-indexer/opensearch-security/roles_mapping.yml
  SUCG: Configuration for 'rolesmapping' created or updated
  SUCG: Expected 1 config types for node {"updated_config_types":["rolesmapping"],"updated_config_size":1,"message":null} is 1 (["rolesmapping"])
  due to: null
Done with success

```

5.5 Tester l'authentification AD

- Déconnecte-toi du Dashboard
- Connecte-toi avec un utilisateur AD (ex: **Administrateur** / mot de passe AD)
- ✓ La connexion doit réussir

6. INSTALLATION DES AGENTS WAZUH

6.1 Principe

Les agents Wazuh sont installés sur les machines à superviser (serveurs, postes). Ils collectent les logs et les envoient au Manager.

6.2 Depuis l'interface Wazuh

- Wazuh Dashboard → **Agents** → **Deploy new agent**
- Choisis le système d'exploitation
- Renseigne l'adresse du Manager : **172.20.1.30**
- Copie la commande générée

6.3 Agent Windows

Installation manuelle :

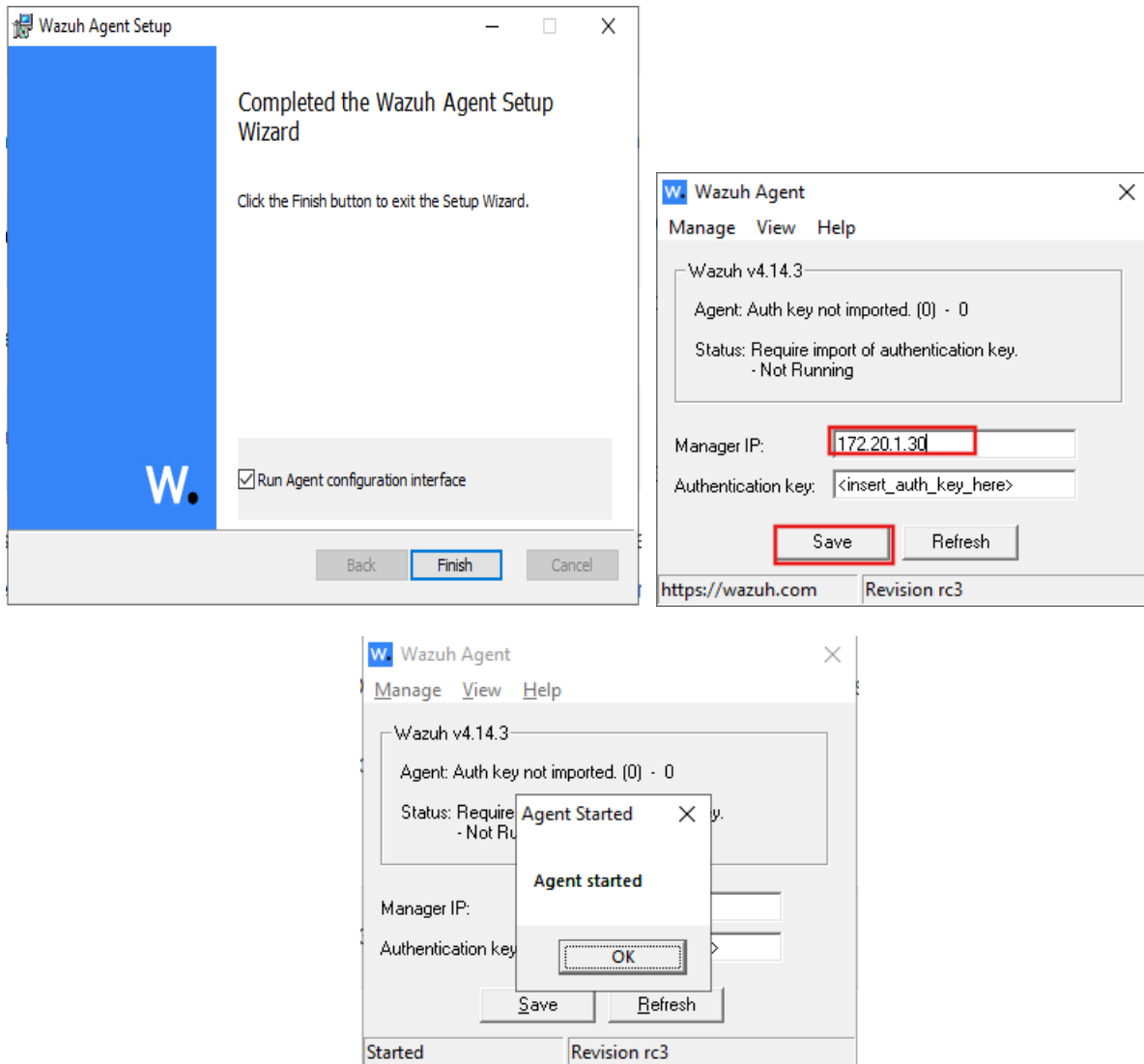
```

# Télécharger l'agent (adapte la version)
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.9.0-1.msi -OutFile ${env:tmp}\wazuh-agent.msi

# Installer avec configuration
msiexec.exe /i ${env:tmp}\wazuh-agent.msi /q WAZUH_MANAGER="172.20.1.30"
WAZUH_REGISTRATION_SERVER="172.20.1.30" WAZUH_AGENT_NAME="hermes-dc"

# Démarrer le service
net start WazuhSvc

```



Vérification :

`Get-Service WazuhSvc`

6.4 Agent Linux

```
# Ajouter la clé GPG
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --dearmor > /usr/share/keyrings/wazuh.gpg
```

```
# Ajouter le dépôt
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | tee /etc/apt/sources.list.d/wazuh.list
```

```
root@ocsjplpi:~# curl -s https://packages.wazuh.com/key/GPG-KFY-WAZUH | gpg --dearmor > /usr/share/keyrings/wazuh.gpg
root@ocsjplpi:~# echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | tee /etc/a
deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main
root@ocsjplpi:~# apt update
WAZUH_MANAGER="172.20.1.30" apt-get install -y wazuh-agent
Atteint :1 http://deb.debian.org/debian bookworm InRelease
Atteint :2 http://deb.debian.org/debian bookworm-updates InRelease
Atteint :3 http://security.debian.org/debian-security bookworm-security InRelease
Réception de :4 https://packages.wazuh.com/4.x/apt stable InRelease [17,3 kB]
Réception de :5 https://packages.wazuh.com/4.x/apt stable/main amd64 Packages [50,6 kB]
67,9 ko réceptionnés en 1s (78,8 ko/s)
Lecture des listes de paquets... Fait
# Mettre à jour et installer
apt update
WAZUH_MANAGER="172.20.1.30" apt-get install -y wazuh-agent

# Démarrer
systemctl daemon-reload
systemctl enable wazuh-agent
systemctl start wazuh-agent
```

6.5 Vérification des agents

- Depuis le Dashboard Wazuh : **Agents** → Tous les agents doivent apparaître avec statut **Active**
- Ou en ligne de commande :

```
/var/ossec/bin/agent_control -l
```

```
root@wazuh:/home/user# /var/ossec/bin/agent_control -l
Wazuh agent_control. List of available agents:
  ID: 000, Name: wazuh (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: hermes, IP: any, Active
  ID: 002, Name: Ares, IP: any, Disconnected
  ID: 003, Name: neptune2, IP: any, Disconnected
  ID: 004, Name: nagios, IP: any, Active
  ID: 005, Name: ocsjplpi.stadiumcompany.local, IP: any, Active

List of agentless devices:
root@wazuh:/home/user#
```

7. CONFIGURATION DES RÈGLES PERSONNALISÉES

7.1 Créer des règles pour détecter des événements AD

```
nano /var/ossec/etc/rules/local_rules.xml
```

```
<group name="windows,authentication,">

  <!-- Détection de verrouillage de compte (Event ID 4740) -->
  <rule id="100001" level="10">
    <if_sid>60003</if_sid>
    <match>Event ID: 4740</match>
    <description>Compte utilisateur verrouillé (Event ID 4740)</description>
    <options>no_full_log</options>
    <group>account_lock,</group>
```

```

</rule>

<!-- Détection de création de compte (Event ID 4720) -->
<rule id="100002" level="8">
  <if_sid>60003</if_sid>
  <match>Event ID: 4720</match>
  <description>Nouveau compte utilisateur créé</description>
  <group>account_creation,</group>
</rule>

<!-- Détection d'ajout à un groupe de sécurité (Event ID 4732) -->
<rule id="100003" level="12">
  <if_sid>60003</if_sid>
  <match>Event ID: 4732</match>
  <description>Membre ajouté à un groupe de sécurité</description>
  <group>group_change,</group>
</rule>

<!-- Détection de suppression de compte (Event ID 4726) -->
<rule id="100004" level="9">
  <if_sid>60003</if_sid>
  <match>Event ID: 4726</match>
  <description>Compte utilisateur supprimé</description>
  <group>account_deletion,</group>
</rule>

<!-- Détection de modification de mot de passe (Event ID 4723) -->
<rule id="100005" level="7">
  <if_sid>60003</if_sid>
  <match>Event ID: 4723</match>
  <description>Tentative de changement de mot de passe</description>
  <group>password_change,</group>
</rule>

</group>

```

```
<group name="windows,authentication,">

  <!-- Détection de verrouillage de compte (Event ID 4740) -->
  <rule id="100001" level="10">
    <if_sid>60003</if_sid>
    <match>Event ID: 4740</match>
    <description>Compte utilisateur verrouillé (Event ID 4740)</description>
    <options>no_full_log</options>
    <group>account_lock,</group>
  </rule>

  <!-- Détection de création de compte (Event ID 4720) -->
  <rule id="100002" level="8">
    <if_sid>60003</if_sid>
    <match>Event ID: 4720</match>
    <description>Nouveau compte utilisateur créé</description>
    <group>account_creation,</group>
  </rule>

  <!-- Détection d'ajout à un groupe de sécurité (Event ID 4732) -->
  <rule id="100003" level="12">
    <if_sid>60003</if_sid>
    <match>Event ID: 4732</match>
    <description>Membre ajouté à un groupe de sécurité</description>
    <group>group_change,</group>
  </rule>

  <!-- Détection de suppression de compte (Event ID 4726) -->
  <rule id="100004" level="9">
    <if_sid>60003</if_sid>
    <match>Event ID: 4726</match>
    <description>Compte utilisateur supprimé</description>
    <group>account_deletion,</group>
  </rule>

  <!-- Détection de modification de mot de passe (Event ID 4723) -->
  <rule id="100005" level="7">
    <if_sid>60003</if_sid>
    <match>Event ID: 4723</match>
    <description>Tentative de changement de mot de passe</description>
    <group>password_change,</group>
  </rule>
</group>
```

7.2 Redémarrer Wazuh

```
systemctl restart wazuh-manager
```

7.3 Vérifier que les règles sont chargées

```
tail -f /var/ossec/logs/ossec.log | grep -i "rule"
```

8. TESTS ET VALIDATION

8.1 Test d'authentification Active Directory

1. **Déconnecte-toi** du Dashboard Wazuh (icône en haut à droite → *Logout*).
2. **Connecte-toi** avec un utilisateur Active Directory valide, par exemple :
 - **Nom d'utilisateur** : cesar
 - **Mot de passe** : Bts2024@
3. ✓ **Vérifie** que le nom de l'utilisateur AD apparaît en haut à droite du Dashboard.
 - Si c'est le cas, l'intégration LDAP/AD fonctionne correctement.

8.2 Test de génération d'alerte (simulation locale)

Sur une machine où l'agent Wazuh est installé (par exemple **Hermès**), exécute ces commandes **PowerShell en administrateur** pour simuler une activité :

```
# Créer un utilisateur local temporaire
$null = New-LocalUser -Name "TestUser" -Password (ConvertTo-SecureString
"Bts2024@" -AsPlainText -Force)
```

```
# Supprimer l'utilisateur local
$null = Remove-LocalUser -Name "TestUser"
```

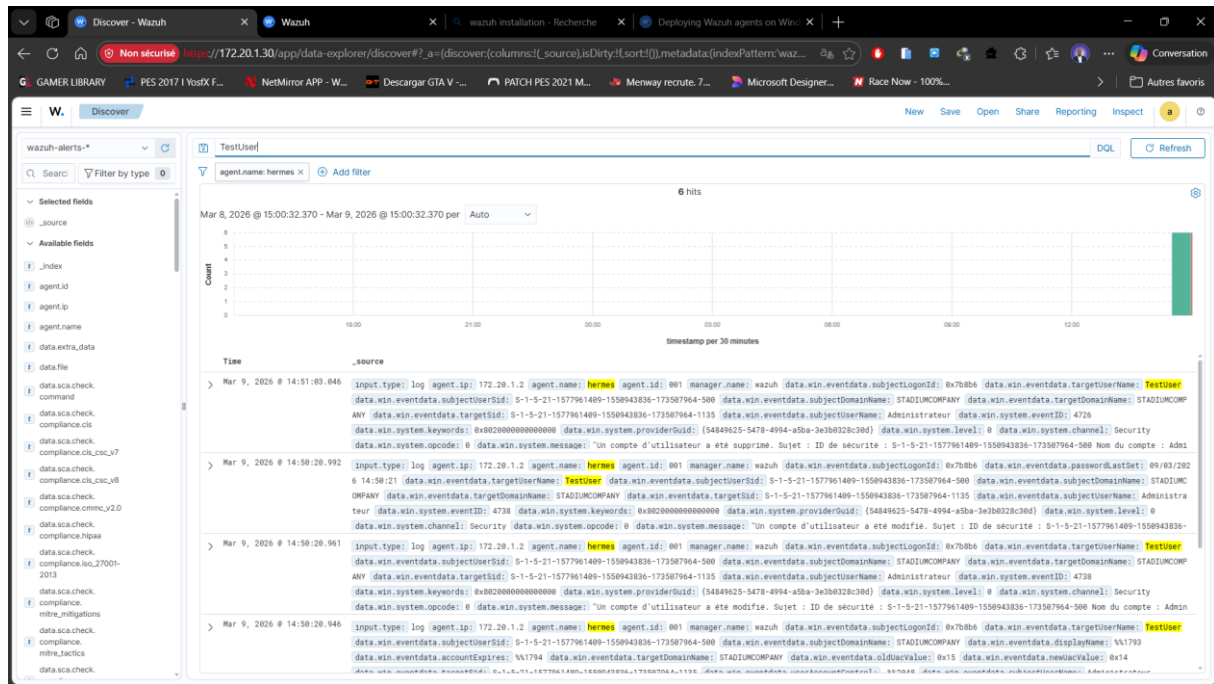
```
PS C:\Users\Administrateur> $null = New-LocalUser -Name "TestUser" -Password (ConvertTo-SecureString
"@azerty1234" -AsPlainText -force)
PS C:\Users\Administrateur> $null = Remove-LocalUser -Name "TestUser"
PS C:\Users\Administrateur>
```

✦ Dans Wazuh Dashboard :

- Va dans l'onglet **Security Events**.
- Applique un filtre sur `agent.name: hermes` (ou le nom de ta machine).
- ✔ **Vérifie** que des événements remontent (création/suppression de compte).

The screenshot shows the Wazuh Dashboard interface. On the left, there's a sidebar with 'wazuh-alerts-*' and a search bar. The main area displays a list of security events filtered by 'agent.name: hermes'. The events are listed in a table with columns for timestamp, agent name, and event details. The events show a sequence of actions related to the creation and deletion of a user named 'TestUser'.

Timestamp	Agent Name	Event Details
Mar 9, 2024 @ 14:54:45.734	hermes	input.type: log agent.ip: 172.20.1.2 agent.name: hermes agent.id: 001 manager.name: wazuh data.win.eventdata.subjectLogonId: 0 data.win.eventdata.targetLogonId: 0 data.win.eventdata.targetLinkedLogonId: 0 data.win.eventdata.imPERSONATIONLevel: V\1633 data.win.eventdata.ipAddress: 127.0.0.1 data.win.eventdata.authenticationPackageName: Kerberos data.win.eventdata.targetLogonId: 0x3de28d data.win.eventdata.logonProcessName: Kerberos data.win.eventdata.logonId: 72726c89-a652-b4b-7773-a938441b3e data.win.eventdata.targetUser: HERMES5 data.win.eventdata.keylength: 0 data.win.eventdata.elevationToken: V\1642 data.win.eventdata.subjectUserSid: 5-1-5-18 data.win.eventdata.processId: 0 data.win.eventdata.ipPort: 56282 data.win.eventdata.targetDomainName: STADIUMCOMPANY.LOCAL data.win.eventdata.targetUserSid: 5-1-5-18 data.win.eventdata.virtualAccount: V\1643 data.win.eventdata.logonType: 3
Mar 9, 2024 @ 14:53:45.666	hermes	input.type: log agent.ip: 172.20.1.2 agent.name: hermes agent.id: 001 manager.name: wazuh data.win.eventdata.subjectLogonId: 0 data.win.eventdata.targetLogonId: 0x3de28d data.win.eventdata.targetLinkedLogonId: 0 data.win.eventdata.imPERSONATIONLevel: V\1633 data.win.eventdata.ipAddress: 127.0.0.1 data.win.eventdata.authenticationPackageName: Kerberos data.win.eventdata.targetLogonId: 0x3de28d data.win.eventdata.logonProcessName: Kerberos data.win.eventdata.logonId: 72726c89-a652-b4b-7773-a938441b3e data.win.eventdata.targetUser: HERMES5 data.win.eventdata.keylength: 0 data.win.eventdata.elevationToken: V\1642 data.win.eventdata.subjectUserSid: 5-1-5-18 data.win.eventdata.processId: 0 data.win.eventdata.ipPort: 56282 data.win.eventdata.targetDomainName: STADIUMCOMPANY.LOCAL data.win.eventdata.targetUserSid: 5-1-5-18 data.win.eventdata.virtualAccount: V\1643 data.win.eventdata.logonType: 3
Mar 9, 2024 @ 14:53:45.653	hermes	input.type: log agent.ip: 172.20.1.2 agent.name: hermes agent.id: 001 manager.name: wazuh data.win.eventdata.subjectLogonId: 0 data.win.eventdata.targetLogonId: 0x3de28d data.win.eventdata.targetLinkedLogonId: 0 data.win.eventdata.imPERSONATIONLevel: V\1633 data.win.eventdata.ipAddress: 127.0.0.1 data.win.eventdata.authenticationPackageName: Kerberos data.win.eventdata.targetLogonId: 0x3de28d data.win.eventdata.logonProcessName: Kerberos data.win.eventdata.logonId: 72726c89-a652-b4b-7773-a938441b3e data.win.eventdata.targetUser: HERMES5 data.win.eventdata.keylength: 0 data.win.eventdata.elevationToken: V\1642 data.win.eventdata.subjectUserSid: 5-1-5-18 data.win.eventdata.processId: 0 data.win.eventdata.ipPort: 56282 data.win.eventdata.targetDomainName: STADIUMCOMPANY.LOCAL data.win.eventdata.targetUserSid: 5-1-5-18 data.win.eventdata.virtualAccount: V\1643 data.win.eventdata.logonType: 3



8.3 Test de détection spécifique (règles personnalisées AD)

Cette étape permet de valider les règles personnalisées créées dans `local_rules.xml`.

◆ Déclenche un verrouillage de compte AD

Sur **Hermès** (ou tout contrôleur de domaine), exécute la commande suivante pour désactiver temporairement un compte utilisateur (ex: kaiser) :

```
# Désactiver le compte (simule un verrouillage)
Disable-ADAccount -Identity "kaiser"
```

```
PS C:\Users\Administrateur> Disable-ADAccount -Identity "kaiser"
PS C:\Users\Administrateur>
```

Tu peux aussi simuler plusieurs échecs de connexion pour provoquer un verrouillage automatique.

◆ Vérification dans Wazuh

- Dans le Dashboard, va dans **Security Events**.
- Recherche l'alerte avec la règle `rule.id: 100001`.
- ✔ L'alerte "**Compte utilisateur verrouillé (Event ID 4740)**" doit apparaître dans les **5 minutes** suivant l'événement.

💡 Si l'alerte n'apparaît pas, vérifie que :

- L'agent sur Hermès est bien actif (`agent_control -l`).
- Les logs Windows remontent correctement.
- La règle a été chargée (`grep 100001 /var/ossec/logs/ossec.log`).

8.4 Test de création de compte AD

Toujours sur Hermès, crée un utilisateur de test dans Active Directory :

```
New-ADUser -Name "TestADUser" -SamAccountName "testad" -UserPrincipalName
"testad@stadiumcompany.local" -Enabled $true -AccountPassword (ConvertTo-
SecureString "Bts2024@" -AsPlainText -Force)
```

Puis recherche dans Wazuh Dashboard l'alerte correspondante à la règle `rule.id: 100002` (création de compte).

8.5 Test d'ajout à un groupe privilégié

Ajoute l'utilisateur `testad` au groupe **Domain Admins** :

```
Add-ADGroupMember -Identity "Domain Admins" -Members testad
```

Recherche dans Wazuh Dashboard l'alerte correspondant à la règle `rule.id: 100003` (ajout à un groupe de sécurité).